21 Python for AI/ML Interview Questions

The Most Important Python Skills Every AI/ML Engineer Must Know

NumPy | Pandas | OOP | Preprocessing | Performance | Coding Challenges

Python Core for ML	**NumPy Essentials**	**Pandas for ML**
Data Preprocessing	**OOP & Advanced Python**	**File I/O & Performance**
	Coding Challenges	

AmanAI Lab

amanailab.com | YouTube: @AmanAILab

FREE RESOURCE - Share with your network!

Table of Contents

Python Core for ML

Q1 What is the difference between list, tuple, set, and dictionary? When do you use each in ML?

Answer:

List: Ordered, mutable, allows duplicates. Use for: storing sequences of data points, feature lists, batch predictions. **Tuple**: Ordered, immutable. Use for: returning multiple metrics from functions, dictionary keys, model hyperparameters that shouldn't change. **Set**: Unordered, no duplicates. Use for: finding unique categories, deduplication, fast O(1) membership checking ('is this word in vocabulary?'). **Dictionary**: Key-value pairs. Use for: model configs, feature mappings, label encoders, JSON responses. Key performance note: 'if x in my_list' is O(n) — slow. 'if x in my_set' is O(1) — fast. For vocabulary checks in NLP, always use a set.

■ **Code Example:**

```
# List - store predictions
predictions = [0.92, 0.87, 0.45, 0.99]

# Tuple - return multiple metrics (immutable)
def evaluate(y_true, y_pred):
return (precision, recall, f1) # can't accidentally modify

# Set - unique labels (O(1) lookup)
labels = ['cat', 'dog', 'cat', 'bird']
unique = set(labels) # {'cat', 'dog', 'bird'}
if 'cat' in unique: # O(1) - instant
pass

# Dictionary - model config
config = {'lr': 0.001, 'batch_size': 32, 'epochs': 100}
model = XGBClassifier(**config) # unpack dict into kwargs
```

■ *Interview Tip: Mention O(1) vs O(n) lookup difference between set and list. Say 'For NLP vocabulary checks with 50K words, I always use a set — membership testing is O(1) instead of O(n) with a list.' Shows performance awareness.*

Q2 Explain list comprehension vs generator expressions. When do you use each in ML pipelines?

Answer:

List comprehension [x for x in data]: Creates full list in memory at once. Fast, readable, supports indexing, can iterate multiple times. **Generator expression** (x for x in data): Creates values one at a time

on-demand. Memory efficient — only one value in memory at a time. Can only iterate ONCE. Use list comprehension when: data is small-medium, you need indexing, iterating multiple times. Use generators when: data is large (millions of rows), feeding into model training, processing images/audio that don't fit in RAM. In ML: generators are critical for **data loading** — you can't load 10GB of images into memory. You generate batches one at a time during training.

■ **Code Example:**

```
# List - all in memory (10GB of images!)
features = [extract(img) for img in all_images] # DANGEROUS for large data

# Generator - one batch at a time
def batch_generator(images, batch_size=32):
for i in range(0, len(images), batch_size):
batch = images[i:i+batch_size]
yield [extract(img) for img in batch] # only 32 images in memory

# Training loop:
for batch in batch_generator(training_images, 32):
model.train(batch) # 32 images at a time, not all 100K

# Quick feature engineering (small data):
normalized = [x / max_val for x in raw_features] # list comp is fine
log_feats = [math.log(x+1) for x in raw_features]

# Conditional:
high_conf = [p for p in predictions if p > 0.5]
```

■ *Interview Tip: Say 'For data loading in ML, I always use generators because datasets often don't fit in memory. For feature engineering on small arrays, list comprehensions for readability.' Shows you pick the right tool.*

Q3 What are *args and **kwargs? How are they used in ML code?

Answer:

***args** collects any number of positional arguments into a tuple. ****kwargs** collects any number of keyword arguments into a dictionary. They make functions flexible — accepting arguments without defining each one explicitly. In ML code: (1) **Config-driven model creation** — unpack a config dict into model parameters: XGBClassifier(**config). (2) **Model factory functions** — create different models with different parameters using the same function. (3) **Decorators** — timing and retry decorators that work with ANY function regardless of its arguments. (4) **Wrapper functions** — pass through all arguments when wrapping sklearn/PyTorch classes.

■ **Code Example:**

```
# Config-driven model (most common ML use case)
config = {'n_estimators': 200, 'max_depth': 6, 'learning_rate': 0.01}
model = XGBClassifier(**config) # unpacks dict into keyword arguments

# Model factory
def create_model(model_class, **kwargs):
return model_class(**kwargs)

rf = create_model(RandomForestClassifier, n_estimators=100, max_depth=5)
xgb = create_model(XGBClassifier, learning_rate=0.01, n_estimators=200)

# Timer decorator - works with ANY function
import time, functools
def timer(func):
@functools.wraps(func)
def wrapper(*args, **kwargs): # accepts any args
start = time.time()
result = func(*args, **kwargs)
print(f'{func.__name__}: {time.time()-start:.2f}s')
return result
return wrapper

@timer
def train(X, y, epochs=10): pass # timer works on any function
```

■ *Interview Tip: Show the config-driven pattern: model = XGBClassifier(**config). Say 'In production, model configs come from YAML files, not hardcoded parameters. **kwargs makes this seamless.' This is how real production ML code works.*

NumPy Essentials

Q4 Why is NumPy 100x faster than Python lists for numerical operations?

Answer:

Three fundamental reasons: (1) **Contiguous memory** — NumPy stores elements in a continuous block of memory like C arrays. Python lists store pointers to objects scattered randomly in memory. Contiguous memory means better CPU cache utilization. (2) **Vectorized operations in C** — NumPy operations are implemented in C and use SIMD (Single Instruction Multiple Data) CPU instructions to process multiple elements simultaneously. Python loops process one element at a time in interpreted Python. (3) **Fixed data type** — NumPy arrays have ONE dtype (float64, int32). No type checking per element. Python lists can mix types, requiring type checking on every operation. Result: NumPy is typically 10-100x faster for numerical operations. Always use NumPy vectorized operations instead of Python loops in ML.

■ **Code Example:**

```
import numpy as np, time

size = 1_000_000

# Python list (slow)
py_list = list(range(size))
start = time.time()
result = [x * 2 for x in py_list]
print(f'Python: {time.time()-start:.4f}s') # ~0.080s

# NumPy (fast)
np_arr = np.arange(size)
start = time.time()
result = np_arr * 2 # vectorized, no loop!
print(f'NumPy: {time.time()-start:.4f}s') # ~0.002s

# NumPy is ~40x faster!

# ML impact: normalizing 1M features:
# Python: [x/max_val for x in features] -> 0.08s
# NumPy: features / max_val -> 0.002s
# In training (1000 batches): 80s vs 2s total

# RULE: Never use Python loops for numerical ML operations
# SLOW: for i in range(n): result[i] = data[i] * 2
# FAST: result = data * 2
```

■ *Interview Tip: Give the three reasons in order: contiguous memory, vectorization in C, fixed type. Then say 'I never use Python loops for numerical operations — always NumPy vectorized.' Shows performance-first thinking.*

Q5 What is broadcasting in NumPy and where do you use it in ML?

Answer:

Broadcasting is NumPy's ability to perform operations on arrays of different shapes by automatically 'stretching' the smaller array to match the larger one — without copying data. Rules: (1) If arrays have different number of dimensions, pad the smaller one with 1s on the LEFT. (2) Arrays with size 1 along a dimension are stretched to match. (3) If sizes don't match and neither is 1 — error. In ML: broadcasting is used EVERYWHERE — feature normalization (subtract mean, divide by std across all rows), adding bias vectors in neural networks, computing pairwise distances. It's the core mechanism that makes vectorized ML code possible.

■ **Code Example:**

```
import numpy as np

# ML Use Case 1: Feature normalization
data = np.array([[1, 200, 50],
[2, 150, 30],
[3, 300, 70]]) # shape (3, 3)

mean = data.mean(axis=0) # shape (3,) - one mean per column
std = data.std(axis=0) # shape (3,)
normalized = (data - mean) / std # (3,3) - (3,) -> BROADCASTS!
# Each column gets its own mean/std applied to all 3 rows

# ML Use Case 2: Add bias in neural network
weights = np.random.randn(64, 128) # (64, 128)
bias = np.random.randn(128) # (128,)
output = weights + bias # broadcasts (128,) across all 64 rows

# ML Use Case 3: Threshold predictions
predictions = np.array([0.92, 0.45, 0.88, 0.31]) # (4,)
threshold = 0.5 # scalar
binary = predictions > threshold # [True, False, True, False]
```

■ *Interview Tip: When asked about broadcasting, trace the shapes: (3,3) - (3,) -> works. (3,3) - (2,) -> error. Always draw shape diagrams. Interviewers love when you can trace shapes mentally.*

Q6 What is the difference between NumPy array copy and view? Why does it matter in ML?

Answer:

View: A new array object that SHARES the same data in memory. Modifying the view MODIFIES the original. Created by: slicing (arr[1:5]), reshape(), transpose (.T). **Copy**: Completely independent array with its own data. Modifying copy does NOT affect original. Created by: .copy(), fancy indexing arr[[0,2,4]], boolean indexing arr[arr>0]. Why it matters in ML: (1) **Data corruption** — if you slice training data and normalize it in place, you accidentally corrupt the original dataset. All subsequent batches get already-normalized data, normalized again — wrong! (2) **Memory efficiency** — views are free (O(1), no copy), copies cost memory proportional to data size. Use views when reading, copies when you need to modify.

■ **Code Example:**

```
import numpy as np

# VIEW danger:
X_train = np.array([[1,2,3],[4,5,6],[7,8,9]]) # original data
batch = X_train[0:2] # VIEW! shares memory
batch = batch / 255.0 # THIS MODIFIES X_train!
print(X_train[0]) # [0.004, 0.008, 0.012] CORRUPTED!

# COPY - safe:
batch = X_train[0:2].copy() # independent copy
batch = batch / 255.0 # X_train is unchanged
print(X_train[0]) # [1, 2, 3] SAFE!

# Check if it's a view:
print(batch.base is X_train) # True = view, None = copy

# When to use each:
# Reading only -> use view (free)
row_view = X_train[0] # just reading -> no need for copy

# Modifying -> always copy
batch_copy = X_train[0:32].copy() # will normalize
batch_copy /= 255.0 # safe
```

■ *Interview Tip: Mention data corruption as the real risk. 'I always use .copy() when preprocessing batches to avoid accidentally modifying the original training data. This is a subtle bug that's very hard to debug in production.'*

Pandas for ML

Q7 What is the difference between apply(), map(), and applymap() in Pandas?

Answer:

map(): Works on a **Series only**. Applies function or dictionary mapping to each element. Best for: simple element-wise transformations, category replacements. **apply()**: Works on **both Series and DataFrame**. On DataFrame: axis=0 applies to each column, axis=1 applies to each row. Most versatile. **applymap() / map() on DataFrame**: Element-wise on entire DataFrame. Best for type conversion across all cells. **CRITICAL performance rule**: Vectorized Pandas operations are always faster than apply(). apply() is 10-100x slower because it runs Python code row-by-row instead of using optimized C code. Use apply() only when vectorization is genuinely not possible.

■ **Code Example:**

```
import pandas as pd, numpy as np

df = pd.DataFrame({'dept': ['eng','sales','eng'], 'salary':
[50000,60000,70000]})

# map() - Series only, element-wise replacement
df['dept'] = df['dept'].map({'eng': 'Engineering', 'sales': 'Sales'})

# apply() on Series
df['tax'] = df['salary'].apply(lambda x: x * 0.3)

# apply() on DataFrame - row-wise (axis=1)
df['info'] = df.apply(lambda r: f"{r['dept']}: ${r['salary']}", axis=1)

# apply() on DataFrame - column-wise (axis=0)
df[['salary','tax']].apply(np.mean, axis=0)

# ALWAYS prefer vectorized over apply():
# SLOW: df['tax'] = df['salary'].apply(lambda x: x * 0.3) # Python loop
# FAST: df['tax'] = df['salary'] * 0.3 # vectorized C code (100x faster)

# SLOW: df['log_sal'] = df['salary'].apply(np.log)
# FAST: df['log_sal'] = np.log(df['salary'])
```

■ *Interview Tip: Say 'I always try vectorized operations first because they are 100x faster than apply(). I only use apply() when the logic genuinely cannot be vectorized.' Shows you understand Pandas performance.*

Q8 How do you handle missing values in Pandas for ML without causing data leakage?

Answer:

Missing data handling has a critical rule: **NEVER fit on test data**. Strategy: (1) **Detect**: df.isnull().sum() for counts, df.isnull().mean() for percentages. (2) **Analyze the pattern**: Is it random (MCAR), depends on other columns (MAR), or depends on the missing value itself (MNAR)? (3) **Strategy per column**: Numerical with few missing → median imputation (robust to outliers). Categorical → mode or 'Unknown'. >50% missing → consider dropping. (4) **Add missing indicator**: Create binary 'column_was_missing' — the fact that data is missing can be informative. (5) **Use sklearn SimpleImputer**: fit() on train only, transform() on both train and test. This prevents leakage.

■ **Code Example:**

```
import pandas as pd, numpy as np
from sklearn.impute import SimpleImputer

df = pd.DataFrame({'age': [25, np.nan, 35], 'salary': [50000, 60000, np.nan],
'city': ['NYC', np.nan, 'LA']})

# Step 1: Add missing indicator BEFORE imputing
df['age_missing'] = df['age'].isnull().astype(int)

# Step 2: Impute with sklearn (correct, no leakage)
num_imputer = SimpleImputer(strategy='median')
cat_imputer = SimpleImputer(strategy='most_frequent')

# Fit ONLY on training data:
num_imputer.fit(X_train[['age', 'salary']])
cat_imputer.fit(X_train[['city']])

# Transform both train and test with TRAIN statistics:
X_train[['age','salary']] = num_imputer.transform(X_train[['age','salary']])
X_test[['age','salary']] = num_imputer.transform(X_test[['age','salary']])

# WRONG - data leakage! (uses test data stats)
# df['age'].fillna(df['age'].mean()) # mean includes test data!
# imputer.fit(X) # sees test data!
```

■ *Interview Tip: Data leakage is the #1 answer interviewers want here. Say 'I always fit the imputer on training data only. Fitting on the full dataset leaks test statistics into the model, making evaluation misleadingly optimistic.' This separates you from 90% of candidates.*

Q9 Explain groupby() with a real ML feature engineering example.

Answer:

groupby() splits the DataFrame by column values, applies a function to each group, and combines results. It's the foundation of **aggregated feature engineering** in ML — capturing entity-level behavior patterns from transaction/event data. Key difference: **agg()** reduces rows (one row per group), **transform()** keeps the same number of rows as original (injects group statistics back into the original DataFrame). In ML: groupby creates features that capture user/store/product behavior — critical for fraud detection, churn prediction, recommendation systems.

■ **Code Example:**

```
import pandas as pd

# Transaction data
df = pd.DataFrame({
'user_id': [1,1,1,2,2,3,3,3],
'amount': [100,200,150,300,50,80,90,200],
'category':['food','tech','food','tech','food','food','tech','food']
})

# agg() - user-level features (reduces to 1 row per user)
user_feats = df.groupby('user_id')['amount'].agg(
avg_spend='mean',
total_spend='sum',
num_transactions='count',
spend_std='std'
).reset_index()

# transform() - inject group stats back into original df (same shape!)
df['user_avg_spend'] = df.groupby('user_id')['amount'].transform('mean')
df['spend_vs_user_avg'] = df['amount'] - df['user_avg_spend'] # deviation

# Multi-level: spending per user per category
category_pivot =
df.groupby(['user_id','category'])['amount'].sum().unstack(fill_value=0)
# Creates: user_id | food_spend | tech_spend

# These behavioral features are what makes fraud detection work!
```

■ *Interview Tip: Show transform() vs agg() difference. 'agg() reduces rows — one per group. transform() keeps the same number of rows so I can add group statistics as new features without merging.' This shows you understand the practical difference.*

Data Preprocessing

Q10 How do you detect and handle outliers in Python for ML?

Answer:

Three detection methods: (1) **IQR method** — Q1, Q3, IQR=Q3-Q1. Outliers below Q1-1.5*IQR or above Q3+1.5*IQR. Simple, robust, no distribution assumption. (2) **Z-score** — values with |z| > 3 are outliers. Assumes normal distribution. (3) **Isolation Forest** — ML-based, works for multivariate outliers, no distribution assumption. Handling strategies: (a) **Remove** — if clearly an error (negative age). (b) **Cap/Winsorize** — clip to 1st/99th percentile. (c) **Log transform** — reduces extreme value impact. (d)

Keep — if outliers are meaningful signals (in fraud detection, the outlier IS the fraud!). Key: understand WHY before deciding what to do.

■ Code Example:

```
import numpy as np, pandas as pd

df = pd.DataFrame({'salary': [30000,35000,40000,50000,1000000]})

# IQR method
Q1, Q3 = df['salary'].quantile([0.25, 0.75])
IQR = Q3 - Q1
lower = Q1 - 1.5 * IQR # 18750
upper = Q3 + 1.5 * IQR # 67500
outliers = df[(df['salary'] < lower) | (df['salary'] > upper)]
# Detects: 1000000

# Z-score method
from scipy import stats
df['z'] = np.abs(stats.zscore(df['salary']))
outliers = df[df['z'] > 3]

# Handling 1: Cap at 99th percentile (Winsorize)
cap = df['salary'].quantile(0.99)
df['salary_capped'] = df['salary'].clip(upper=cap)

# Handling 2: Log transform
df['salary_log'] = np.log1p(df['salary']) # log(1+x), handles zeros

# Isolation Forest (multivariate)
from sklearn.ensemble import IsolationForest
iso = IsolationForest(contamination=0.05)
df['outlier'] = iso.fit_predict(df[['salary']]) # -1 = outlier
```

■ *Interview Tip: Always ask 'What is the business context?' In salary data, $1M might be an error. In fraud detection, $1M IS what you're looking for. Context determines the strategy — this shows business thinking.*

Q11 How do you encode categorical variables in Python for ML?

Answer:

Four encoding methods for different scenarios: (1) **One-Hot Encoding** — binary column per category. Use for: nominal data (no order), linear models, neural networks. Problem: explodes dimensions with high-cardinality features (10K cities = 10K columns). (2) **Ordinal Encoding** — assign meaningful integer order. Use for: inherently ordered data (small=1, medium=2, large=3). (3) **Label Encoding** — arbitrary integer per category. Use for: tree-based models only (they don't assume order). NOT for linear models. (4) **Target/Mean Encoding** — replace category with mean of target. Use for: high-cardinality features.

Must use cross-validation regularization to prevent overfitting. Always: fit encoder on training data only, apply to test.

■ Code Example:

```
import pandas as pd
from sklearn.preprocessing import OneHotEncoder, OrdinalEncoder

# One-Hot (nominal categories)
enc = OneHotEncoder(sparse_output=False, handle_unknown='ignore')
enc.fit(X_train[['color']]) # fit on TRAIN only
X_train_enc = enc.transform(X_train[['color']])
X_test_enc = enc.transform(X_test[['color']]) # apply train mapping

# Or with pandas:
df = pd.get_dummies(df, columns=['color'], drop_first=True)
# drop_first avoids multicollinearity

# Ordinal (ordered categories)
size_map = {'S': 1, 'M': 2, 'L': 3, 'XL': 4}
df['size_enc'] = df['size'].map(size_map)

# Target encoding (high-cardinality like city with 10K values)
city_means = X_train.groupby('city')['target'].mean()
X_train['city_enc'] = X_train['city'].map(city_means)
X_test['city_enc'] = X_test['city'].map(city_means) # train means only!

# High-cardinality problem:
# 'city' with 10,000 unique values -> one-hot = 10,000 columns!
# Solution: target encoding or frequency encoding
```

■ *Interview Tip: Mention the high-cardinality problem. 'One-hot encoding a city column with 10K unique values creates 10K columns — the curse of dimensionality. I use target encoding with cross-validation for high-cardinality features.' Shows practical production experience.*

Q12 What is the difference between fit(), transform(), and fit_transform()? Why does it prevent data leakage?

Answer:

fit(): Learns parameters FROM the data (e.g., mean and std for StandardScaler). ONLY call on training data. **transform()**: APPLIES the learned parameters to data. Call on both train and test using the same learned parameters. **fit_transform()**: Convenience method = fit() + transform() in one step. Use ONLY on training data. The CRITICAL rule: fitting on test data causes **data leakage** — the model indirectly sees test data statistics during training, producing misleadingly optimistic evaluation scores that don't reflect real production performance. In production, you'll have worse accuracy than your evaluation suggested.

■ **Code Example:**

```
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2)
scaler = StandardScaler()

# CORRECT: Fit on train only
X_train_scaled = scaler.fit_transform(X_train) # learns mean/std from train
X_test_scaled = scaler.transform(X_test) # applies TRAIN's mean/std to test

# What fit() actually learns:
# scaler.mean_ = [mean of each feature in X_train]
# scaler.scale_ = [std of each feature in X_train]
# These exact values are applied to X_test

# WRONG (data leakage!):
scaler.fit(X) # sees ALL data including test!
X_train_scaled = scaler.transform(X_train)

# ALSO WRONG:
X_test_scaled = scaler.fit_transform(X_test) # learns NEW params from test!

# Best practice: use sklearn Pipeline (prevents leakage automatically)
from sklearn.pipeline import Pipeline
pipeline = Pipeline([('scaler', StandardScaler()), ('model', model)])
pipeline.fit(X_train, y_train) # safe - pipeline never leaks
```

■ *Interview Tip: This is a DATA LEAKAGE question. Say 'Fitting on test data is the most common mistake in ML — it inflates evaluation metrics. I use sklearn Pipeline which handles this automatically, ensuring the scaler is only fit on training data.' Mention Pipeline.*

OOP & Advanced Python

Q13 How do you create custom sklearn transformers using OOP for ML pipelines?

Answer:

Custom transformers allow you to plug your own feature engineering into sklearn Pipeline — getting all the benefits of Pipeline (no data leakage, easy cross-validation, single joblib save). To create one: inherit from **BaseEstimator** and **TransformerMixin**. Implement **fit(X, y=None)** — learn any statistics from training data. Implement **transform(X)** — apply the transformation. The fit/transform separation is critical:

statistics learned in fit() are applied consistently in transform(), preventing data leakage exactly like StandardScaler.

■ Code Example:

```
from sklearn.base import BaseEstimator, TransformerMixin
import numpy as np

class OutlierClipper(BaseEstimator, TransformerMixin):
def __init__(self, lower=1, upper=99):
self.lower = lower
self.upper = upper

def fit(self, X, y=None):
# Learn percentiles from TRAINING data only
self.lower_ = np.percentile(X, self.lower, axis=0)
self.upper_ = np.percentile(X, self.upper, axis=0)
return self

def transform(self, X):
# Apply TRAINING percentiles to any data (train or test)
return np.clip(X, self.lower_, self.upper_)

# Use in sklearn Pipeline:
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.ensemble import RandomForestClassifier

pipeline = Pipeline([
('clip', OutlierClipper(lower=1, upper=99)),
('scale', StandardScaler()),
('model', RandomForestClassifier(n_estimators=100))
])

pipeline.fit(X_train, y_train) # all steps fit together, no leakage
preds = pipeline.predict(X_test) # all steps applied consistently
```

■ *Interview Tip: Say 'I create custom transformers by inheriting from BaseEstimator and TransformerMixin so they plug into sklearn Pipeline. The fit/transform separation ensures my custom preprocessing has the same data leakage protection as built-in sklearn transformers.'*

Q14 What are decorators in Python and how do you use them in ML code?

Answer:

A **decorator** wraps a function to add behavior without modifying the function itself. Syntax: @decorator above the function. Common ML decorators: (1) **@timer** — measure execution time of training, inference, data loading. (2) **@retry with exponential backoff** — retry API calls (LLM, embedding APIs)

on failure. Essential for production LLM apps. (3) **@lru_cache** — cache expensive computations (feature extraction, embedding lookups). (4) **@validate_input** — check data shape, types, ranges before processing. Key pattern: decorators separate cross-cutting concerns (timing, retry, logging) from business logic (training, prediction).

■ Code Example:

```
import time, functools

# Timer decorator
def timer(func):
@functools.wraps(func)
def wrapper(*args, **kwargs):
start = time.time()
result = func(*args, **kwargs)
print(f'{func.__name__}: {time.time()-start:.2f}s')
return result
return wrapper

@timer
def train_model(X, y): pass
@timer
def generate_embeddings(texts): pass
# Both print timing automatically

# Retry for LLM API calls (most important in production!)
def retry(max_attempts=3, delay=1):
def decorator(func):
def wrapper(*args, **kwargs):
for attempt in range(max_attempts):
try: return func(*args, **kwargs)
except Exception:
if attempt < max_attempts-1:
time.sleep(delay * 2**attempt) # 1s, 2s, 4s
else: raise
return wrapper
return decorator

@retry(max_attempts=3)
def call_openai(text): return client.embeddings.create(input=text)

# Cache expensive computations
from functools import lru_cache
@lru_cache(maxsize=1000)
def extract_features(text): return model.encode(text) # cached after first call
```

■ *Interview Tip: Show the @retry decorator with exponential backoff for API calls. Say 'In production LLM apps, I always wrap API calls with retry logic and exponential backoff. APIs fail randomly and without retry, your pipeline crashes on temporary failures.' This shows production maturity.*

Q15 How do you use lambda, map, and filter efficiently in ML data processing?

Answer:

Lambda: Anonymous function for short one-time operations. Best used inside Pandas .apply(), sorting key=, and sklearn FunctionTransformer. **map()**: Applies a function to every element in an iterable. **filter()**: Keeps elements that satisfy a condition. Important performance note: In ML with Pandas and NumPy, **vectorized operations beat map() and filter() significantly**. Use vectorized when possible. Use map/filter for non-vectorizable logic or when working with plain Python lists. The most common ML use of lambda is inside Pandas apply() for complex row-wise transformations and model factory patterns with **kwargs.

■ **Code Example:**

```
# Lambda - most common ML uses
import pandas as pd, numpy as np
from sklearn.preprocessing import FunctionTransformer

# In Pandas apply
df['text_len'] = df['text'].apply(lambda x: len(x.split()))
df['is_long'] = df['text_len'].apply(lambda x: 1 if x > 100 else 0)

# Config-driven model creation
config = {'lr': 0.01, 'epochs': 100}
model = create_model(**config)

# sklearn FunctionTransformer
log_transform = FunctionTransformer(lambda X: np.log1p(X))

# map() - transform elements
texts = ['Hello World', 'ML IS GREAT']
lower = list(map(str.lower, texts)) # ['hello world', 'ml is great']

# filter() - keep elements
scores = [0.92, 0.45, 0.88, 0.31]
high = list(filter(lambda x: x > 0.7, scores)) # [0.92, 0.88]

# PREFER vectorized over map/filter:
# SLOW: list(map(lambda x: x*2, array)) # Python map
# FAST: array * 2 # NumPy vectorized

# SLOW: list(filter(lambda x: x > 0, series)) # Python filter
# FAST: series[series > 0] # Pandas boolean indexing
```

■ *Interview Tip: Say 'I use lambda primarily inside Pandas apply() for complex transformations that can't be vectorized, and in model factory patterns. For numerical operations, I always prefer vectorized NumPy/Pandas over Python map() or filter().'*

File I/O & Performance

Q16 How do you efficiently read large CSV files in Python for ML?

Answer:

Large CSV files (>1GB) require smart loading strategies: (1) **Select only needed columns**: pd.read_csv(usecols=['col1','col2']). A 50-column CSV where you need 5? 90% memory saved instantly. (2) **Optimize data types**: float64→float32 saves 50%, int64→int32 saves 50%, string→category for low-cardinality saves 90%. (3) **Chunked reading**: pd.read_csv(chunksize=50000) processes the file in chunks. Never loads the whole file. (4) **Convert to Parquet**: Do this once. Parquet is columnar format — 10x faster reads, 60-80% smaller. For any project with large data, converting to Parquet is the #1 ROI optimization. (5) **Polars**: Rust-based DataFrame library, 5-10x faster than Pandas for large files.

■ **Code Example:**

```
import pandas as pd

# Method 1: Column selection + dtype optimization (easiest win)
df = pd.read_csv('data.csv',
usecols=['user_id', 'amount', 'category'], # 3 of 50 columns
dtype={'user_id': 'int32', 'amount': 'float32', 'category': 'category'}
)
# Memory: ~5x less than default

# Method 2: Chunked reading (process huge files)
results = []
for chunk in pd.read_csv('huge_file.csv', chunksize=50000):
processed = chunk[chunk['amount'] > 0] # filter
results.append(processed.groupby('user_id')['amount'].sum())
final = pd.concat(results).groupby(level=0).sum()

# Method 3: Convert to Parquet (one-time, read forever fast)
df = pd.read_csv('data.csv') # slow, one time
df.to_parquet('data.parquet') # save
df = pd.read_parquet('data.parquet') # 10x faster from now on

# Memory comparison (10M rows, 20 columns):
# CSV float64: ~1.6 GB
# CSV float32: ~0.8 GB
# Parquet: ~0.2 GB
```

■ *Interview Tip: Say 'The first thing I do with any new CSV dataset is convert it to Parquet. One-time conversion gives 10x faster reads and 80% less storage for every subsequent read.' This is what production data engineers do.*

Q17 How do you call LLM/embedding APIs efficiently in Python for ML pipelines?

Answer:

Efficient API calling is critical in GenAI pipelines. Three levels of optimization: (1) **Session reuse**: Use requests.Session() to reuse TCP connections. 3-5x faster than creating new connections per call. (2) **Async concurrent calls**: aiohttp + asyncio.gather() for calling APIs concurrently. 10-20x faster for batch operations (embedding 10K documents). (3) **Retry with exponential backoff**: APIs fail randomly. Always retry 3 times with 1s, 2s, 4s delays. (4) **Batch API endpoints**: OpenAI and Anthropic offer batch endpoints at 50% discount for non-real-time work. (5) **Rate limiting**: Respect API rate limits. Use a semaphore to limit concurrent requests.

■ **Code Example:**

```
import asyncio, aiohttp, time, requests

# Basic with retry (minimum for production):
def embed_text(text, max_retries=3):
for attempt in range(max_retries):
try:
r = requests.post(api_url, headers=headers, json={'input': text}, timeout=30)
r.raise_for_status()
return r.json()
except Exception:
if attempt < max_retries-1:
time.sleep(2**attempt) # 1s, 2s, 4s
else: raise

# Session reuse for multiple calls (3-5x faster):
session = requests.Session()
session.headers.update(headers)
for text in texts:
result = session.post(api_url, json={'input': text})

# Async for batch embedding (10x+ faster):
async def embed_all(texts, max_concurrent=10):
sem = asyncio.Semaphore(max_concurrent) # rate limiting
async def embed_one(text):
async with sem:
async with aiohttp.ClientSession() as s:
async with s.post(api_url, json={'input': text}) as r:
return await r.json()
return await asyncio.gather(*[embed_one(t) for t in texts])

results = asyncio.run(embed_all(all_texts)) # 10K texts concurrently
```

■ *Interview Tip: Say 'For batch embedding 10K documents, I use aiohttp with asyncio.gather() and a Semaphore for rate limiting. This is 10-20x faster than sequential requests and critical for production RAG pipelines.' Shows production GenAI experience.*

Q18 What is the difference between multiprocessing and multithreading in Python? When do you use each in ML?

Answer:

Python has the **GIL (Global Interpreter Lock)** — only one thread executes Python code at a time. This fundamentally shapes the choice: **Multithreading** (threading, ThreadPoolExecutor): Multiple threads share the same memory. GIL is released during I/O waits. Best for: **I/O-bound tasks** — API calls, file reading, database queries, downloading data. **Multiprocessing** (multiprocessing, ProcessPoolExecutor): Separate processes with separate memory and separate GILs. Each process uses a different CPU core.

Best for: **CPU-bound tasks** — image preprocessing, feature engineering, data augmentation. Rule: Calling 1000 APIs? → Threading. Processing 10000 images? → Multiprocessing.

■ Code Example:

```
from concurrent.futures import ThreadPoolExecutor, ProcessPoolExecutor

# I/O-bound: Threading for API calls
def call_embedding_api(text):
return requests.post(url, json={'input': text}).json()

texts = ['text1', 'text2', ..., 'text1000']
with ThreadPoolExecutor(max_workers=10) as executor:
embeddings = list(executor.map(call_embedding_api, texts))
# 10 concurrent API calls -> ~10x faster

# CPU-bound: Multiprocessing for image processing
def process_image(path):
img = load_image(path)
return extract_features(img) # CPU-intensive

paths = ['img1.jpg', ..., 'img10000.jpg']
with ProcessPoolExecutor(max_workers=8) as executor:
features = list(executor.map(process_image, paths))
# 8 CPU cores in parallel -> ~8x faster

# PyTorch DataLoader (best for ML training):
from torch.utils.data import DataLoader
loader = DataLoader(dataset, batch_size=32, num_workers=4) # 4 processes
for batch in loader:
model.train_step(batch) # data loaded in parallel with GPU training
```

■ *Interview Tip: Mention the GIL explicitly. Say 'Python GIL prevents true parallelism for CPU tasks in threads, so I use multiprocessing for data preprocessing and threading for API calls. For PyTorch, DataLoader(num_workers=4) handles parallel data loading automatically.'*

Coding Challenges

Q19 Write precision, recall, and F1-score from scratch in Python.

Answer:

This is a classic ML coding interview question — you MUST implement these without sklearn. **Precision** = TP/(TP+FP): Of all predicted positives, how many are actually positive? High precision = few false

alarms. **Recall** = TP/(TP+FN): Of all actual positives, how many did we find? High recall = few misses. **F1** = 2*(P*R)/(P+R): Harmonic mean, balances both. Always handle division by zero. Know WHEN to use each: Precision when false positives are costly (spam filter — don't block legitimate email). Recall when false negatives are costly (cancer detection — don't miss sick patients).

■ Code Example:

```
def compute_metrics(y_true, y_pred):
tp = sum(1 for t,p in zip(y_true,y_pred) if t==1 and p==1)
fp = sum(1 for t,p in zip(y_true,y_pred) if t==0 and p==1)
fn = sum(1 for t,p in zip(y_true,y_pred) if t==1 and p==0)
tn = sum(1 for t,p in zip(y_true,y_pred) if t==0 and p==0)

precision = tp/(tp+fp) if (tp+fp) > 0 else 0
recall = tp/(tp+fn) if (tp+fn) > 0 else 0
f1 = 2*precision*recall/(precision+recall) if (precision+recall) > 0 else 0
accuracy = (tp+tn)/(tp+fp+fn+tn)

return {'precision': round(precision,4), 'recall': round(recall,4),
'f1': round(f1,4), 'accuracy': round(accuracy,4)}

# Test:
y_true = [1,0,1,1,0,1,0,0,1,1]
y_pred = [1,0,1,0,0,1,1,0,1,0]
result = compute_metrics(y_true, y_pred)
print(result) # {'precision': 0.8, 'recall': 0.6667, 'f1': 0.7273, 'accuracy':
0.7}

# Verify:
from sklearn.metrics import precision_score
print(precision_score(y_true, y_pred)) # 0.8 - matches!
```

■ *Interview Tip: Handle division by zero — many candidates forget and get ZeroDivisionError. Also say WHEN to use each: 'Precision for spam (false positives costly), Recall for medical diagnosis (false negatives costly), F1 when both matter equally.'*

Q20 Write k-fold cross-validation from scratch in Python.

Answer:

K-fold cross-validation splits data into K equal folds. Each fold takes a turn as the test set (K times). The remaining K-1 folds train the model. You get K accuracy scores — report mean and standard deviation. Why it's better than one train-test split: a single split might be lucky or unlucky. K splits give K evaluations, showing both performance (mean) and stability (std). Low std = consistent model. High std = model is sensitive to which data it trains on. Key details: shuffle before splitting (avoid order bias), use stratified splits for imbalanced classes (keep class proportions equal in each fold).

■ **Code Example:**

```
import numpy as np

def k_fold_cv(X, y, model_class, k=5, shuffle=True, **model_params):
n = len(X)
if shuffle:
idx = np.random.permutation(n)
X, y = X[idx], y[idx]

fold_size = n // k
scores = []

for i in range(k):
# Test indices for this fold
start = i * fold_size
end = start + fold_size if i < k-1 else n

X_test = X[start:end]
y_test = y[start:end]
X_train = np.concatenate([X[:start], X[end:]])
y_train = np.concatenate([y[:start], y[end:]])

model = model_class(**model_params)
model.fit(X_train, y_train)
scores.append(model.score(X_test, y_test))

return {'mean': np.mean(scores), 'std': np.std(scores), 'scores': scores}

# Usage:
from sklearn.ensemble import RandomForestClassifier
result = k_fold_cv(X, y, RandomForestClassifier, k=5, n_estimators=100)
print(f'Accuracy: {result["mean"]:.4f} +/- {result["std"]:.4f}')
# Accuracy: 0.8920 +/- 0.0134
```

■ *Interview Tip: Mention why cross-validation matters. 'A single train-test split gives one number that could be lucky. K-fold gives K numbers, so I can report mean and standard deviation — showing both performance AND stability of the model.'*

Q21 Write a complete sklearn ML pipeline in Python — preprocessing, training, and evaluation.

Answer:

This is the 'put it all together' question. A production-quality pipeline must: (1) Handle mixed column types (numerical and categorical separately). (2) Use Pipeline to prevent data leakage. (3) Cross-validate for reliable evaluation. (4) Evaluate on held-out test set. (5) Save the complete pipeline for deployment. The

key: **ColumnTransformer** applies different preprocessing to different column types. **Pipeline** chains everything together with automatic leakage prevention. When you call pipeline.fit(X_train), EVERY step fits on training data only. When you call pipeline.predict(X_test), EVERY step applies the training parameters.

■ Code Example:

```
from sklearn.pipeline import Pipeline
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.impute import SimpleImputer
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import train_test_split, cross_val_score
from sklearn.metrics import classification_report
import joblib

num_cols = ['age', 'salary']
cat_cols = ['city', 'dept']

# Preprocessing for each column type
num_pipe = Pipeline([('imputer', SimpleImputer(strategy='median')),
('scaler', StandardScaler())])
cat_pipe = Pipeline([('imputer', SimpleImputer(strategy='most_frequent')),
('encoder', OneHotEncoder(handle_unknown='ignore'))])
preprocessor = ColumnTransformer([('num', num_pipe, num_cols),
('cat', cat_pipe, cat_cols)])
# Full pipeline
pipeline = Pipeline([('prep', preprocessor),
('model', RandomForestClassifier(n_estimators=100))])
# Train and evaluate
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2)
cv = cross_val_score(pipeline, X_train, y_train, cv=5, scoring='f1')
print(f'CV F1: {cv.mean():.4f} +/- {cv.std():.4f}')
pipeline.fit(X_train, y_train)
print(classification_report(y_test, pipeline.predict(X_test)))
joblib.dump(pipeline, 'pipeline.joblib') # save preprocessing + model
```

■ *Interview Tip: This is the PERFECT answer to 'write me an ML pipeline.' Practice writing it from memory in under 10 minutes. The interviewer will notice: ColumnTransformer for mixed types, Pipeline for no leakage, cross-validation, joblib save. All 4 together = senior engineer answer.*

You Made It! ■

You have mastered 21 Python interview questions
every AI/ML engineer must know.

■ Subscribe to AmanAI Lab on YouTube

■ Download 30 LLM Interview Questions PDF

■ Download 30 RAG Interview Questions PDF

■ Download 30 AI Agent Interview Questions PDF

■ Download 30 GenAI Project Questions PDF

amanailab.com

Built with ■ by AmanAI Lab | 2026